IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Un botones virtual que aprende probando

Aprendizaje por refuerzo: recompensas, tabla Q, epsilon-greedy y política

Sprint 9 · S09_04 · cuaderno: 25 min en clase (ejercicios 1 y 2 y entrenamiento) · demostración guiada

LA IDEA · APRENDER PROBANDO

La tercera familia: premios y castigos

Supervisado

Hay respuesta correcta (**cancelada**): el modelo aprende a predecirla.

No supervisado

No hay respuesta: el algoritmo descubre grupos o ejes (S09_01 a S09_03).

Por refuerzo

No hay respuesta correcta: un **agente** prueba, recibe **recompensas** y aprende qué le conviene.

Pieza	En el hotel
Agente y entorno	el botones virtual y el pasillo del hotel
Estado	la casilla donde está (20 posibles)
Acción	arriba, abajo, izquierda o derecha
Recompensa	−1 por paso, −5 en obras, +10 al llegar a la habitación
Política	qué hacer en cada casilla
Episodio	una partida: de recepción a la habitación (o 50 pasos)

EL ENTORNO · SECCIÓN 1

El pasillo del hotel: 4 × 5 casillas



Salida: casilla 0 (recepción) · el número pequeño es el estado

Tres pasos de demostración

derecha → casilla 1 · derecha → casilla 2 · abajo → **choca con la pared y sigue en la 2**. Cada paso: **−1**.

Dos caminos de 7 pasos

Por las obras: −7 − 5 + 10 = **−2**.

Rodeándolas: −7 + 10 = **+3**.

Meta de la partida: llegar a la habitación con la mayor recompensa total, no solo llegar.

LA TABLA Q · EJERCICIO 1 · CUADERNO (CLASE)

Q: cuánto espero ganar si hago esto aquí

Q[estado, acción] empieza en **ceros** (20 filas × 4 columnas). Tras cada paso se corrige un poco hacia lo que **ahora** se cree que vale esa acción.

Ejemplo de una actualización	
Q[s, a] antes	2,00
recompensa r del paso	−1
mejor futuro: max Q[s']	5,00
objetivo = r + gamma · max Q[s'] (gamma 0,95)	3,75
Q nuevo = 2,00 + alpha · (3,75 − 2,00) (alpha 0,2)	2,35

alpha (0 a 1)

Velocidad de aprendizaje: alto se cree lo último; bajo cambia despacio.

gamma (0 a 1)

Cuánto importa el futuro. Con 0 solo mira la recompensa inmediata.

```
objetivo = r + gamma * Q[s2].max()          # lo que ahora crees que vale la acción
Q[s, a] += alpha * (objetivo - Q[s, a])     # mueve Q una fracción alpha hacia ese valor
```

EXPLORAR O EXPLOTAR · EJERCICIO 2 · CUADERNO (CLASE)

A veces, probar algo al azar

epsilon	Qué hace el agente	Consecuencia
0,0	siempre la mejor acción que conoce	nunca descubre caminos mejores
0,2 (por defecto)	1 de cada 5 pasos, al azar	aprende y sigue explorando
1,0	siempre al azar	nunca aprovecha lo aprendido

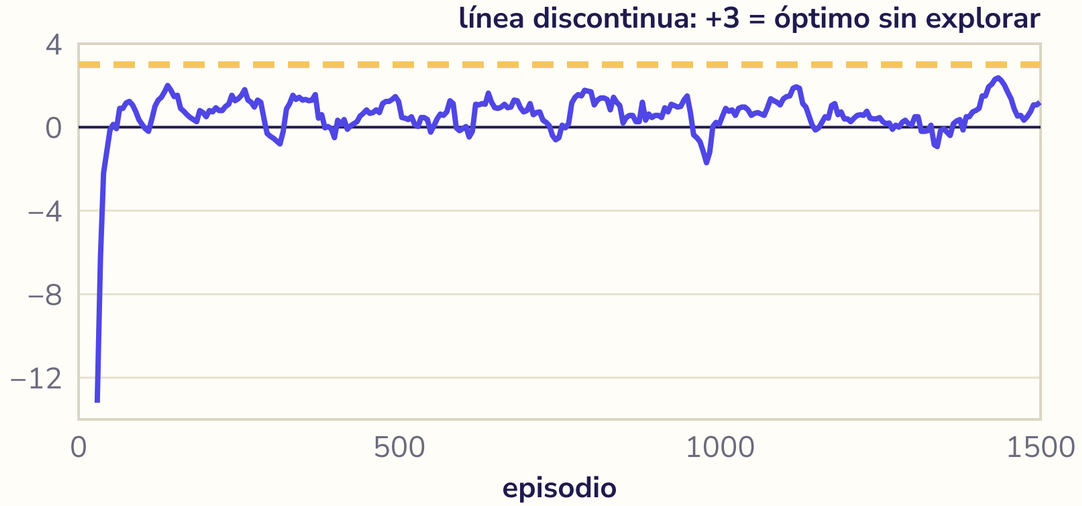
```
if rng.random() < epsilon:
    return int(rng.integers(Q.shape[1]))    # explorar: una acción al azar
return int(np.argmax(Q[s]))                 # explotar: la mejor de la fila s
```

Epsilon-greedy: casi siempre lo mejor conocido, y de vez en cuando una prueba.

ENTRENAR AL AGENTE · SECCIÓN 4 · CUADERNO (CLASE)

1500 partidas y la recompensa mejora

Recompensa total por episodio (media móvil de 30)



−8,0 → 0,9

Media de los **primeros 50** y los **últimos 50**.

Desde el episodio 26

La política **sin explorar** ya es la óptima: **+3**.

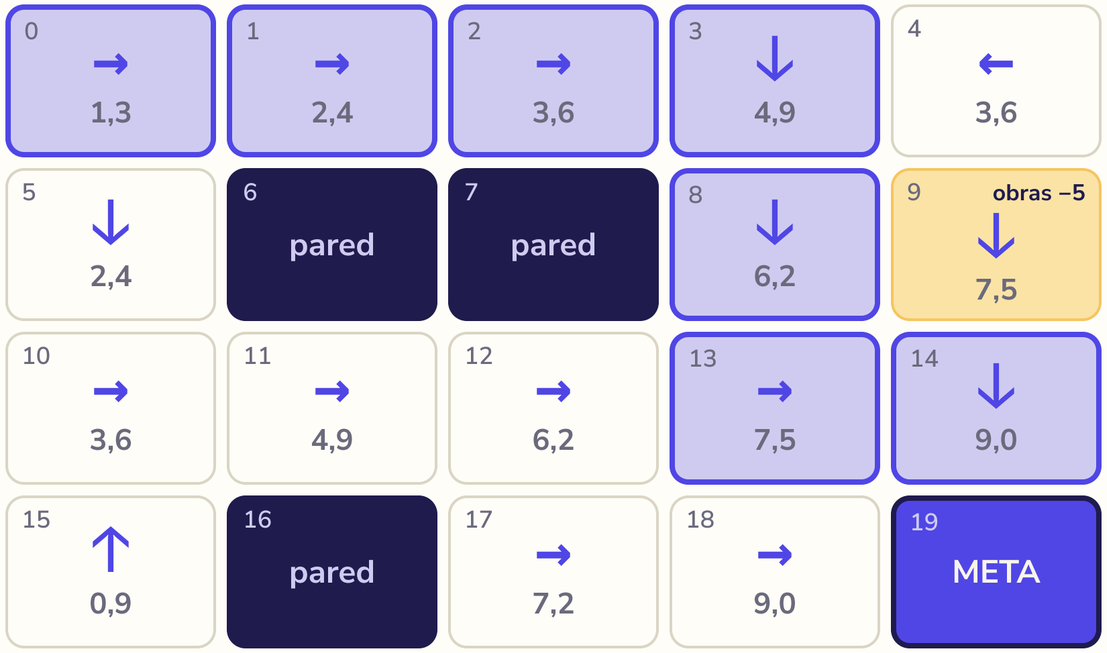
¿Por qué no llega a +3?

Con epsilon 0,2, 1 de cada 5 pasos es al azar.

```
Q, recompensas = entrenar()    # 1500 episodios: alpha 0,2 · gamma 0,95 · epsilon 0,2
```

LEER LA POLÍTICA · EJERCICIO 3 · CUADERNO (CASA)

La mejor acción de cada casilla



Flecha = mejor acción · número = valor de la casilla (max Q) · recuadro = camino aprendido

```
politica = np.argmax(Q, axis=1)

camino = [0, 1, 2, 3, 8, 13, 14, 19]
```

7 pasos y +3

Evita las obras (casilla 9) y llega a la habitación (casilla 19).

EXPERIMENTO · EJERCICIO 4 · CUADERNO (CASA)

Sin exploración y sin pistas, no aprende

Entrenamiento (1500 episodios)	Recompensa media de los últimos 100
pasos gratis (coste_paso=0), epsilon 0,0	0,0
pasos gratis (coste_paso=0), epsilon 0,3	9,1
coste −1 por paso (por defecto), epsilon 0,0	3,0
gamma=0 (sin mirar al futuro)	−49,5

Pasos gratis

La única recompensa está al final: sin explorar, el agente repite siempre la misma acción y **no encuentra la meta**.

Con coste por paso

Hasta el que no explora aprende: chocar sale caro y le empuja a probar otra dirección (**exploración implícita**).

gamma = 0

Solo ve la recompensa inmediata: −49,5 frente al **+3** óptimo.

MINI-RETO · REFLEXIÓN 4 (CASA)

El agente hace lo que se premia

Cambio de reglas en las obras	Camino aprendido (sin explorar)	Recompensa	¿Llega?
−5 (las reglas originales)	rodea las obras, 7 pasos	+3	sí
−20 (reclamaciones)	el mismo camino	+3	sí
+5 (una «recompensa» por pasar)	llega a las obras y **se queda en ellas**	+180	no

Una recompensa mal diseñada produce un comportamiento que no queremos.

Precios dinámicos

Un agente que maximiza ingresos: ¿qué recompensa mal diseñada perjudicaría a los clientes o a la reputación del hotel?

HOY HAS APRENDIDO

Un agente que aprende probando

Agente y recompensa

Estado, acción, recompensa, política y episodio: las piezas de todo problema de refuerzo

Q-learning

La tabla Q se corrige tras cada paso: **alpha** es la velocidad y **gamma**, el peso del futuro

Explorar y diseñar

Sin explorar no descubre; y el agente hace lo que se **premia**, no lo que queremos

Siguiente: el reto del Sprint 9 y, después, redes neuronales (Sprint 10)